Distilling and Deploying a Command-Parsing LM on a Local Kubernetes Cloud

NAME, ROLL NUMBER — GitHub: REPO LINK

> **Draft status.** Text marked **[FILL]** must come from *your own* runs. Nothing below is a measurement unless a number is filled in. Delete this box before submitting.

0. Architecture

Ollama (phi3.5, temperature 0, seed 42) labels 3,000 commands in parallel workers (namespace `pipeline`) that read from and write to SeaweedFS (namespace `data`, buckets `datasets`, `models`). `clean.py` filters the replies into `datasets/v1/train.jsonl`. Pythia-70M is fine-tuned on the first 1,500 records (sequence-level distillation) on CPU; weights go to `models/v1`, `models/v2` and a GitHub Release. A FastAPI service in a CPU-only image downloads the model at start-up and is deployed in namespace `app` behind a Service, Traefik Ingress and an HPA. CI (GitHub Actions) lints, tests, evaluates the released model and gates on JSON validity. *Assumption:* Ollama runs on the host, not in the cluster. **[FILL: add a diagram if space allows.]**

1. Task 1 — Object storage and parallel labelling

Setup: `cluster/k3d.yaml`, `k8s/10-seaweedfs.yaml`, `k8s/20-teacher-job.yaml`, `prompts/teacher_prompt.txt`.

Sharding. The worker reads `SHARD_INDEX` and `SHARD_COUNT` and processes the commands whose 0-based line number *n* satisfies *n* mod `SHARD_COUNT` = `SHARD_INDEX`. This is a deterministic partition: each line has exactly one remainder, so every id lands in exactly one shard with no gaps or repeats, and no coordination between workers is needed. Round-robin also balances load if command length varies along the file. An Indexed Kubernetes Job supplies the index through the completion-index annotation, so the two pods differ only in that value. Each worker writes only its own object (`teacher/shard-<i>.jsonl`), so there are no write conflicts, and it checkpoints and resumes after a restart. *Limitation:* the teacher on one laptop CPU is the bottleneck, so two workers share the same Ollama server and the speed-up is limited. **[FILL: confirm what you observed.]**

Counts (measured). [FILL from `clean.py` output]

input	rejected (a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	duplicates	kept

Check: input = a+b+c+d+duplicates+kept. SHA-256 of `train.jsonl`: **[FILL]**.

2. Task 2 — Student training (sequence-level distillation)

Config (`training/config.yaml`): seed, learning rate, batch size, epochs, max length, library versions **[FILL final values]**. Loss only on the teacher's reply tokens. Per-epoch loss in `training/loss.csv` **[FILL values]**. `models/v2` differs by **[FILL: e.g. epochs/seed]**. Model checksum: **[FILL]**.

Results on validation (measured) [FILL]

model	json_validity	intent	date	time	place	full_frame
teacher phi3.5						
Pythia-70M untrained						
student v1						

Discussion [FILL]: what changed between untrained and trained, which slots are hardest, how close the student is to the teacher.

Sequence-level vs logit-based distillation. Sequence-level distillation (Kim & Rush, 2016) trains the student with ordinary cross-entropy on the *text* the teacher generated, so the teacher is only a label generator. Logit-based distillation trains the student to match the teacher's per-token probability distribution (for example with KL divergence), which carries more information than the single chosen token. It is not possible here because the two models use different tokenizers and vocabularies (phi3.5 vs Pythia/GPT-NeoX), so their output distributions cannot be aligned token by token. In addition, Ollama returns text rather than full logits. A limitation of what I did: the student can only be as good as the teacher's labels, including their systematic errors (rules about copying slot text are applied imperfectly by a 3.8B model), and a 70M model trained on at most 1,500 examples will generalise poorly to phrasings it has not seen. **[FILL: replace with a limitation you actually observed.]**

3. Task 3 — Inference service

`service/app.py`: `/parse` (200 with the four-key object plus `X-Model-Version`, `X-Served-By`), `/healthz`, `/readyz` (503 until loaded). Every output is validated; an invalid one gives 500 `{"error": "invalid_model_output"}`; `FAULT_INJECT=1` replaces the output with `not json` to test this. Config comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`; credentials from a Kubernetes Secret. Image: pinned dependencies, CPU-only torch, no weights. Image size: **[FILL]** MB. 50 sequential requests at `--cpus=1 --memory=1g`: OOM-killed? **[FILL]**; peak memory **[FILL]** (command output in `results/`).

4. Task 4 — Kubernetes deployment, scaling, reliability

Manifests: `k8s/30-app.yaml` (2 replicas, requests 500m/512Mi, limits 1 CPU/1Gi, startup/liveness/readiness probes, HPA 1–2 at 70% CPU, `maxUnavailable: 0`).

experiment	measured result
Service has no ready endpoints while loading	**[FILL]**
HPA scale 1→2 under load, back to 1	**[FILL]**
Rolling update v1→v2: failed requests / total	**[FILL]** (target ≤0.5%)
After `rollout undo`: `X-Model-Version`	**[FILL]**
Pod deleted: replicas restored; failed %	**[FILL]** (target ≤2%)
Cold start, 3 runs and median	**[FILL]** s
Load balancing: requests per replica	**[FILL]** (each ≥30%)

Limitations of the local cluster. The cluster is a single k3d node on one laptop, so there is no fault tolerance across machines: a node or laptop failure takes everything down. The HPA's second replica competes for the same CPU as the first and as the load generator, so scaling gains are limited. Ollama and the load test share the host. Metrics-server sampling delays make the HPA react slowly. **[FILL: add any you hit.]**

5. Task 5 — Subnetting and NACL

Subnet	Network	Prefix	Usable range	Broadcast	Usable hosts
app	10.20.0.0	/22	10.20.0.1–10.20.3.254	10.20.3.255	1022
public	10.20.4.0	/24	10.20.4.1–10.20.4.254	10.20.4.255	254
data	10.20.5.0	/25	10.20.5.1–10.20.5.126	10.20.5.127	126
management	10.20.5.128	/27	10.20.5.129–10.20.5.158	10.20.5.159	30

`network/nacl.json` passes 6/6 sample packets in `nacl_sim.py` (measured). A NACL is stateless, so it keeps no record of connections: a reply is a separate packet checked on its own and needs its own rule (outbound 1024–65535 to clients; inbound 1024–65535 from the app subnet). Rules run from the lowest number and the first match wins, so the deny for 198.51.100.0/24 is rule 90, below the allows on 80/443; with a higher number the allow would match first and the block would never apply.

6. Task 6 — Reproducibility, testing, CI/CD

`make up` / `make down`; workflow `.github/workflows/ci.yml`: lint → unit tests (4) → evaluation of the Release model → gate `--min-validity 95`; gitleaks scan. Failed run with the untrained model (workflow_dispatch, `model=base`): **[FILL link]**. Secret scan result: **[FILL]**. AI use: see `AI_USE.md`.

7. Task 7 — Performance, cost, review

Locust (4 users, 2 min, through the Ingress) **[FILL]**: p50 ___ ms, p95 ___ ms, ___ req/s, error rate ***%, kubectl top pods*** , cold start ___ s (Task 4).

Cost (`python scripts/cost.py`, rates from `rate_card.json`: 3.50 per vCPU-hour, 0.45 per GiB-hour): hourly cost = replicas × (CPU limit × 3.50 + memory limit GiB × 0.45); requests/hour = measured req/s × 3600; cost per 1000 = hourly / requests per hour × 1000. With 2 replicas at 1 CPU/1 GiB the hourly cost is 2 × (3.50 + 0.45) = **7.90 INR**. Plug in your measured req/s: **[FILL]**.

SLO. "99% of requests succeed during one load-test run." Error budget = 1% × total requests = **[FILL]** requests; used: **[FILL]** failed requests = ___% of budget.

Review [FILL every measured cell]

Component	Evidence (measured)	Limitation found	Improvement
Data and storage			
Model quality			
Latency and throughput			
Resource utilisation			
Reliability and recovery			
Security and cost			